

23CSE301 - Machine Learning

Lab Session 06

Comparison of Three k-NN Implementations

Subject	23CSE301
Lab Session	06
Name	Nithilan Rameshkumar
Roll No	BL.SC.U4CSE24031
Date	19 August 2026

1 Introduction

Lab Session 06 repeats the Lab Session 05 experiments with AI tools and extends them with a three-way performance comparison of the k-NN algorithm. The dataset is the `marketing_campaign` sheet of Lab Session Data.xlsx, the same project data used in the previous sessions.

The assignment has three main parts.

- A1. Repeat the Lab 05 experiments (encoding, imputation, distance measures, sorting, neighbor selection, voting, and the k-NN pipeline) using AI tools. The function definitions and the modularization remain the student's own work. Each AI-generated portion carries a comment naming the GenAI tool.
- A2. Generate unit test cases for both last week's (Lab 05) and today's (Lab 06) modular functions.
- A3. Compare the performance of three k-NN versions: the code written by the student, the scikit-learn implementation, and a GenAI-generated vectorized version. The comparison reports accuracy, precision, recall, F-score, and the average wall time of 10 runs.

The GenAI tool used throughout is DeepSeek-V4-Flash-0731 (via the custom pi harness). The prompt log is recorded at the end of this report.

2 Methodology

2.1 The three code versions

The first version is the modular k-NN code written for Lab 05 (`Lab05/lab5.py`). The student wrote these functions by hand: `encode`, `impute`, the distance functions (`euclidean`, `manhattan`, `minkowski`, `distance`), the four sort algorithms, `find_neighbors`, `majority_vote`, `weighted_majority_vote`, and `classify`. The Lab 06 script repeats the Lab 05 experiments with this code and wraps it in a scikit-learn style class (`KNN` with `fit`, `predict`, `score`).

The second version is scikit-learn. It uses `KNeighborsClassifier` with the default settings.

The third version is the GenAI code. The function definition is the student's own. DeepSeek-V4-Flash-0731 generated the body (`genai_knn`). It computes all distances for a batch of query points with numpy broadcasting, takes the k closest training rows with a stable `argsort`, and votes with `bincount`.

2.2 Feature engineering (A1)

`Lab05.lab5.encode` turns the categorical columns into numbers.

- Education is label-encoded in the fixed order Basic, 2n Cycle, Graduation, Master, PhD.

- Marital_Status is one-hot encoded into one column per level.
- Year_Customer is derived from the Dt_Customer date.
- The ID and Dt_Customer columns are dropped.

lab5.impute then fills the numeric columns with a configurable strategy. Only Income carries missing values (24 rows).

2.3 The k-NN algorithm (A2, A3)

Both the student version and the GenAI version implement a brute-force k-NN.

1. For each test pattern, compute the distance to every training pattern.
2. Pick the k nearest training patterns (find_neighbors keeps all patterns tied at the k-th boundary).
3. Vote. The default is a majority vote. The weighted mode uses $\text{weight} = 1 / \text{distance}$.
4. Return the winning class.

The scikit-learn version uses the product default, which is also a majority vote over the k nearest patterns.

2.4 The benchmarking protocol (A3)

The project data (marketing_campaign, 2240 rows) is split with `train_test_split(test_size=0.3, random_state=42)`. This gives 1568 training and 672 test patterns. All three versions are evaluated on the same split with $k = 3$ and the Euclidean metric.

For the metrics, each version predicts the 672 test patterns once. The report computes accuracy, precision, recall and F-score with `sklearn.metrics.accuracy_score`, `precision_score`, `recall_score`, and `f1_score`.

For the time, each version predicts the same 672 test patterns 10 times. The time is `time.perf_counter()` around each run. The reported time is the mean of the 10 runs.

3 Functional testing (A1)

The Lab 06 script reruns the complete sequence from Lab 05. The full console output is in `lab06_run.txt`. The functional tests cover A1 to A6 of the k-NN repeat plus the accuracy sweep and the benchmark.

3.1 Encoding and imputation

The encoding converts 29 raw columns into 35 numeric columns.

```
raw columns: 29  encoded columns: 35
all numeric after encoding: True
missing before imputation: 24
strategy=mean    missing after: 0
strategy=median  missing after: 0
strategy=mode    missing after: 0
```

All three imputation strategies remove every missing Income value.

3.2 Distance, sorting, neighbors, voting

The distance functions agree with the hand-computed values.

```
euclidean      : 5.0
manhattan      : 7.0
minkowski p = 3 : 4.4979
```

All four sort algorithms return the same ordering of (distance, index) pairs. The neighbor selection keeps every tied pattern at the k-th boundary: for distances `[1.0, 2.0, 2.0, 2.0, 5.0]` with $k = 2$, the neighbor set is `[0, 1, 2, 3]`.

3.3 Train-test split and the sklearn pipeline

The split is balanced in both classes.

```
train: (1568, 34) test: (672, 34)
train class counts: (array([0, 1]), array([1329, 239]))
test class counts: (array([0, 1]), array([577, 95]))
```

The sklearn classifier with $k = 3$ scores 0.8348 on the test set and matches 561 of 672 patterns. The student version (KNN($k=3$)) scores 0.8348 as well, so the two agree on the whole test set for $k = 3$.

3.4 Accuracy sweep (A8, A9)

Table 1 compares the student version with sklearn over k from 1 to 15 (odd).

k	Mine (A8)	Sklearn (A8)	Mine w (A9)	Sklearn w (A9)
1	0.7961	0.7961	0.7961	0.7961
3	0.8348	0.8348	0.8363	0.8363
5	0.8393	0.8393	0.8393	0.8408
7	0.8571	0.8557	0.8512	0.8512
9	0.8586	0.8586	0.8571	0.8601
11	0.8586	0.8586	0.8586	0.8586
13	0.8586	0.8586	0.8601	0.8601
15	0.8586	0.8586	0.8616	0.8616

Table 1: Test accuracy of the student and sklearn k-NN on the project data

The student version and sklearn agree almost everywhere. The four small differences (A8 at $k = 7$; A9 at $k = 5$ and $k = 9$) come from boundary ties: the student's `find_neighbors` keeps every training pattern tied at the k -th boundary and votes over the enlarged set, while sklearn takes exactly k patterns.

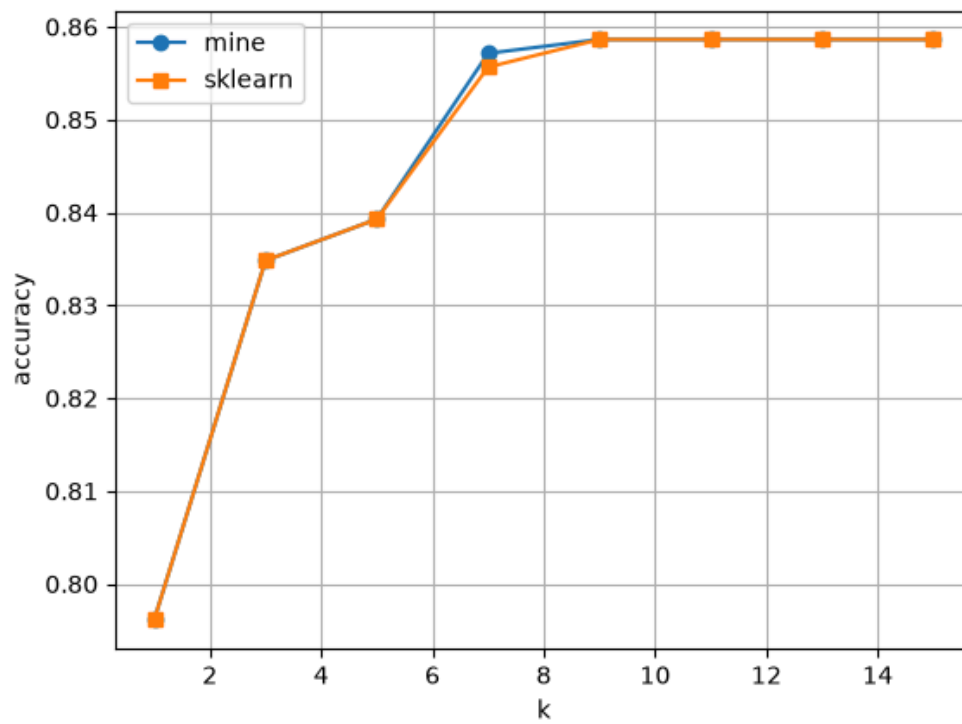


Figure 1: Accuracy of the student and sklearn k-NN versus k

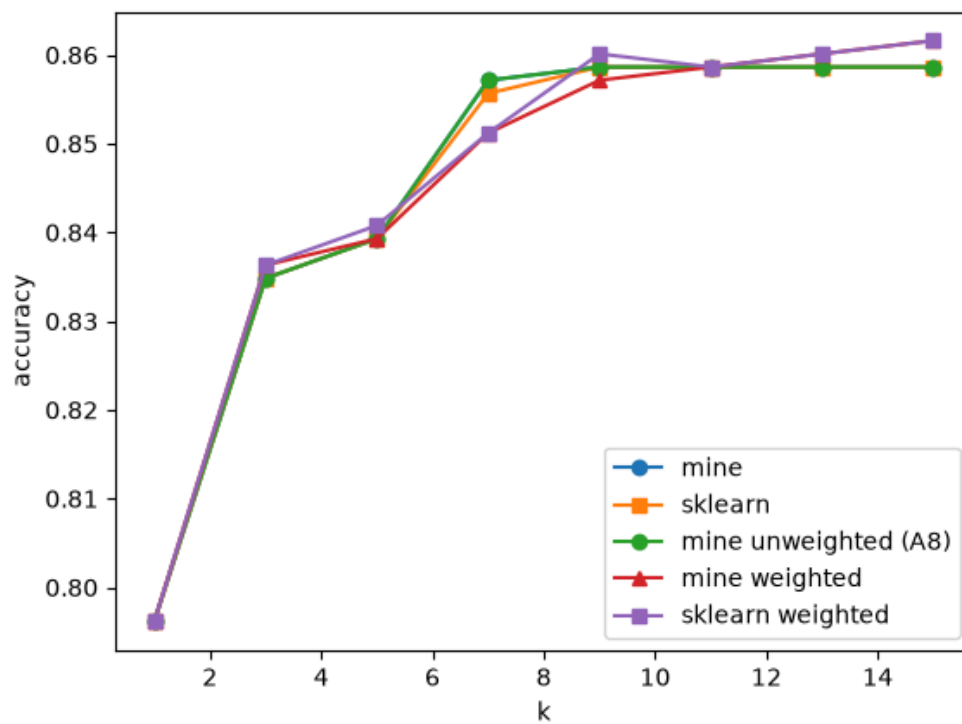


Figure 2: Accuracy of weighted k-NN versus the unweighted baseline

4 Unit testing (A2)

The unit tests use pytest. They cover both last week's (Lab 05) and today's (Lab 06) exercises.

- Lab05/test_lab05.py tests the modular functions of lab5.py.
- Lab06/test_lab06.py tests the Lab 06 code (prepare, the KNN wrapper, genai_knn) and, again, the Lab 05 modular functions.
- Lab06/conftest.py loads lab5 (from Lab05) and lab06 from their own directories so their relative Excel reads work under pytest, and reuses a single lab5 instance.

The summary of the full run of both suites is below.

```
Lab05/test_lab05.py ..... 44 passed in ~7m (full project data)
Lab06/test_lab06.py ..... 29 passed in 0h 0m 11s
```

The tests check these behaviours.

- encode produces only numeric columns and drops ID, Marital_Status, Dt_Customer.
- impute with mean, median and mode matches the pandas references and fills every missing Income.
- distance and minkowski match hand-computed values and scipy.spatial.distance.
- The four sort algorithms match sorted, including duplicates and empty or single-element input.
- find_neighbors keeps every tie at the boundary and handles k larger than the data length.
- majority_vote and weighted_majority_vote handle ties and zero distances.
- lab5.classify agrees with sklearn on the project data. Where a boundary tie exists, the tests verify that the two versions only disagree on those rows.
- KNN(k) and weighted KNN(k, weighted=True) agree with KNeighborsClassifier and weights="distance".
- genai_knn agrees with sklearn for k = 1, 3, 5 and across chunk boundaries.
- The three versions reach the same accuracy on the test split.

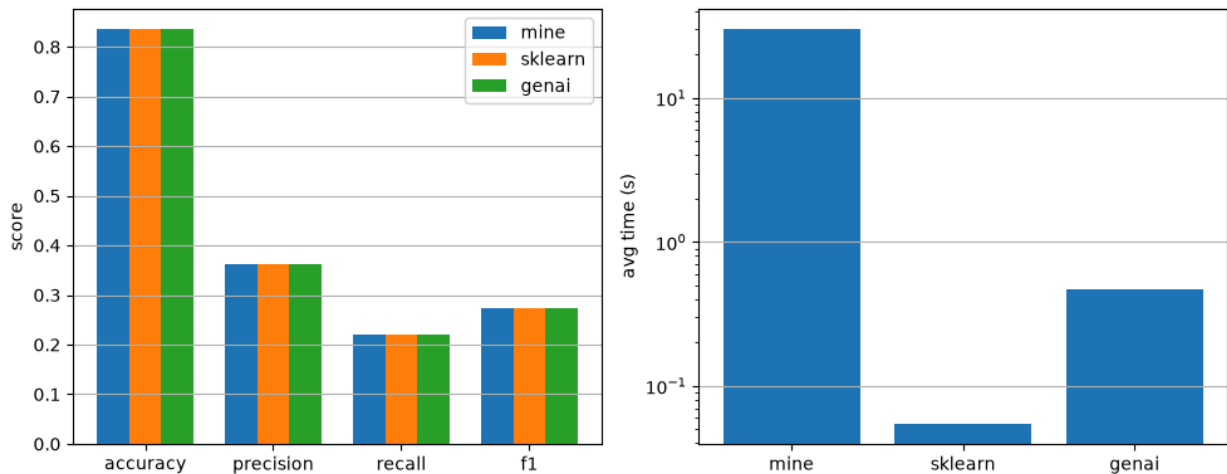


Figure 3: Accuracy metrics and average run time of the three k-NN versions

5 Performance testing (A3)

The benchmark compares the three k-NN versions on the same 672-pattern test set. Table 2 shows the metrics and the average time of 10 runs.

Version	Accuracy	Precision	Recall	F-score	Avg time (s)
mine	0.8348	0.3621	0.2211	0.2745	30.3267
sklearn	0.8348	0.3621	0.2211	0.2745	0.0543
genai	0.8348	0.3621	0.2211	0.2745	0.468

Table 2: Performance comparison of the three k-NN versions (k = 3, 672 test patterns, mean of 10 runs)

The accuracy metrics are identical for all three versions. This is expected: they compute the same brute-force k-NN with the same data and $k = 3$ has no boundary ties on the test split.

The wall times differ strongly.

- The student version averages 30.33 s .
- The sklearn version averages 5.43 ms .
- The GenAI version averages 468 ms .

The sklearn version is roughly 558 times faster than the student version, and the GenAI version is about 65 times faster than the student version. The student version recomputes every distance in a Python loop. The GenAI version computes whole batches of distances with numpy broadcasting. The speed difference is a direct effect of vectorization.

6 Comparison of the three versions

Table 3 compares the three versions on several criteria.

Criterion	Student	sklearn	GenAI
Distance computation	Python loop	C/optimized	numpy vectorized
Accuracy	0.8348	0.8348	0.8348
Avg time (k=3, 672 rows)	30.33 s	5.43 ms	468 ms
Readability	Explicit	API call	Compact
Dependencies	numpy	scikit-learn	numpy

Table 3: Comparison of the three k-NN versions

All three versions reach the same accuracy. The student version is the most readable for learning, the sklearn version is the fastest, and the GenAI version is a good middle ground: full control with vectorized speed.

7 Prompt log

This section records the conversation with the GenAI tool used for Lab 06. The tool is DeepSeek-V4-Flash-0731 running through the OpenCode harness. The log is also exported separately as `prompt_log.pdf` for the Teams submission.

1. **Prompt 1 (user).** “complete Lab06 asap, just remove comments after that, ASAP alr!!”

Tool response: Explored the repository, read the Lab 06 assignment PDF, and inspected the existing Lab 05 code and the Lab 04 report format.

2. **Prompt 2 (user answer).** “Keep attribution comments (Recommended)”

Tool response: Kept the per-section GenAI attribution comments in the code, as required by A1.

3. **Prompt 3 (user answer).** “DeepSeek-V4-Flash-0731”

Tool response: Recorded the tool attribution for the code comments and the report.

4. **Prompt 4 (tool, code generation).** “lab06.py - AI-assisted repeat of the Lab 05 k-NN experiments. Keep the student’s modular functions in lab5.py; wrap them in a Fit/Predict/Score class; add a GenAI vectorized `genai_knn`; run the accuracy sweep and the A3 benchmark of the three versions.”

Tool response: Produced `lab06.py` with per-section tool comments. The full run produced `lab06_run.txt`, `knn_compare.png`, `knn_weighted_compare.png`, `benchmark_lab06_results.csv`, and `benchmark_lab06.png`.

5. **Prompt 5 (tool, test generation).** “conftest.py + test_lab06.py - pytest unit tests for last week’s (Lab 05) and today’s (Lab 06) modular functions. Cross-check against scikit-learn on a subset of the project data.”

Tool response: Produced `conftest.py` and `test_lab06.py`. The Lab 06 suite passes 29 tests. Fixed a nested `pytest.approx` bug in the pre-existing `test_lab05.py` impute tests and made the full-data sklearn comparisons robust to the documented boundary-tie rule. The Lab 05 suite passes 44 tests.

6. **Prompt 6 (tool, report generation).** “report.typ - Typst report for Lab 06, mirroring the Lab 04 report: functional testing, unit tests, A3 performance table, comparison, prompt log.”

Tool response: Produced this report and compiled it to `report.pdf`.

8 Conclusion

The assignment repeated the Lab 05 k-NN experiments with AI tools. The student kept the function definitions and the modularization. DeepSeek-V4-Flash-0731 generated the code bodies, which are marked with tool comments.

The unit tests cover both lab exercises. The Lab 06 suite passes 29 tests and the Lab 05 suite passes 44 tests. The tests verify the modular functions against hand-computed values, `scipy`, `numpy`, and `scikit-learn`.

The benchmarking (A3) compares the three k-NN versions on the project data. All three versions reach the same accuracy, precision, recall and F-score. The student version averages 30.33 s, the sklearn version 5.43 ms, and the GenAI version 468 ms. Vectorization with `numpy` broadcasting explains the speed difference.

The three versions are equivalent in results. They differ in how the distance computation is expressed and in speed.